

CS 330, Fall 2026, Homework 3

Due Wednesday, September 23, 2026

For full credit, you must follow the guidelines laid out on the first page of homework 1.

Problem 1. *The Quickest Round Trip (10 points)*

Your relatives are in town and insist that you take them on a walk through your neighborhood. You really don't want to go as your class is starting soon and you don't want to be late. Because of this you decide to walk the shortest route possible. Since this is sightseeing, you can't walk along the same stretch of road twice. Your goal is to find a route that will get you out and back home as soon as possible without doubling back.

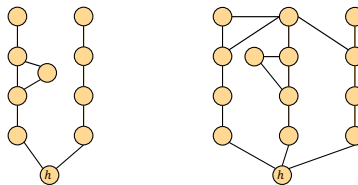
You are given the street map as an **undirected** graph G , where nodes are intersections and edges are roads. As per usual, you may assume that G is given in the form of an adjacency list with the usual implementation. For simplicity, we assume that each street is of equal length and takes the same amount of time to walk. (Thus, G is unweighted.) There is a home node h , from which you depart (along any edge you choose) and to which you must return (along any other edge). A route is valid if it never uses the same edge twice.

Write an algorithm that finds a valid route that uses as few edges as possible.

Inputs The adjacency list of an undirected graph G and home vertex h .

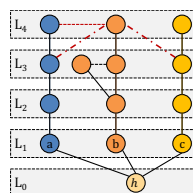
Outputs A Boolean (true/false) indicating whether a valid route from h exists. If one exists, then return the length of the shortest path and a list of nodes on this path. (If there are multiple solutions, you only have to return one.)

For example, in the left-hand graph below, there is no valid route from h . In the right-hand graph, there are three valid routes from h of length 8 (your algorithm just has to find one of them).



For full credit, your algorithm should run in time $O(m + n)$ where $n = |V|$ and $m = |E|$.

Hint: Consider the BFS tree of a graph whose root is h . Moreover, consider subtrees that have each of the immediate children of h as their roots. Observe that an edge connecting two different subtrees creates a valid route. The picture below highlights the different subtrees. The two red edges are part of the shortest cycle(s).



Problem 2. *Your Perfect Train Ride! (10 points)*

You are planning a luxurious train ride across Europe to see the sights of the world. Your first priority for your trip is taking the fewest trains possible. Secondly, you are aware some trains have a reputation for getting crowded, so you would prefer a route with less ruckus. There are lots of different trains and possible ending cities, so you decide to make an algorithm to help yourself decide the perfect train rides to get to any city. The cities will be our nodes, V . The train routes between two cities will be our edges, E . The train system also conveniently has a publicly available hashtable-of-hashtables A where $A.keys = V$ and

$$A[u][v] = (\text{The average number of passengers on the train}),$$

for all pairs (u, v) for which there is a route from u to v . The traffic along each route is symmetric, so that $A[u][v] == A[v][u]$. (That is, the graph is undirected).

For example, consider the graph:

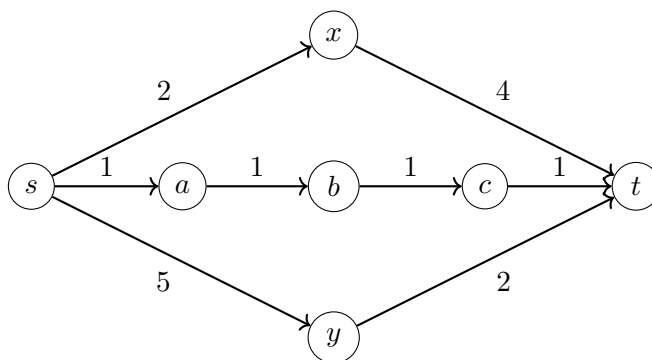


Figure 1: An example graph of trains.

In this graph, if you are traveling from s to t , you would most prefer the route from s to x to t , as that first minimizes the number of train rides for you, and also minimizes the people you interact with among all 2-train routes. Note that the path $s - a - b - c - t$ has lower total weight, but would be less preferred because it has four train rides (edges).

- What algorithm from class would you use to figure out the number of hops you would need to take to get from s to any city? Use that algorithm to compute a hash table *hops*, where $hops[i]$ is a list of all the cities where the minimum number of train rides to get there is i . Explain in one or two sentences how to get *hops*.
- Give an algorithm which takes the *hops* table, as well as A and two nodes s, x as input and returns a best path from s to x as determined by your preferences.